Machine Learning for Finance

A Machine Learning Algorithm for Finite-Horizon Stochastic Control Problems in Economics (2024)

BAGOT Rémi, CHARDIGNY Lola, LETACONNOUX Lauryn

Mastère Spécialisé - Finance and Risk Management
ENSAE - Institut Polytechnique

Avril 2026

Professors: **Jean-David FERMANIAN** and **Huyên PHAM**

Plan

Motivation and problem

The MMCC Algorithm

- Stochastic Control Problems
- MMCC in the time-separable case
- Application to Asian option hedging

Specification of the Application

- State variables and hedging objective
- Discrete-time MMCC formulation
- Neural-network architecture and implementation
- Numerical Setup
- Training Procedure

Results

- Out-of-Sample MMCC Performance
- Convergence Interpretation
- Initial Price and Initial Hedge
- Evolution of the Hedging Strategy
- Convergence Interpretation ($T = 1$ month)

Main motivation

Standard dynamic programming becomes difficult for many finite-horizon stochastic control problems in economics and finance.

- **Finite horizon time inhomogeneity**
- **High dimensional state space**
- **Non time-separable objectives:** Bellman equations may be unavailable or inconvenient.

MMCC idea: simulate trajectories, approximate policies directly with neural networks, and update controls sequentially backward in time.

The MMCC Algorithm

Stochastic Control Problems

Problem 1: time-separable case

$$\max_{(c_0,\theta_1,\ldots,\theta_{T-1})\in\Theta} \quad \mathbb{E}_0\left[\sum_{t=0}^{T-1} u_{t+1}(s_{t+1}, s_t, c_t) \,\middle|\, c_0, \theta_1, \ldots, \theta_{T-1}\right]$$

$$\text{s.t.} \quad c_t = c(t, s_t, \theta_t), \qquad t = 1, \ldots, T-1,$$

$$s_{t+1} = \psi_{t+1}(s_t, c_t, z_{t+1}), \qquad 0 \le t \le T-1, \qquad s_0 \text{ is given.}$$

- We focus on the time-separable case, which corresponds to the first MMCC algorithm in the paper.
- This is the algorithm implemented in our code and presented next.

Problem 2: non time-separable case

$$\max_{(c_0,\theta_1,\ldots,\theta_{T-1})\in\Theta} \quad \mathbb{E}_0\left[u(s_0, c_0, s_1, c_1, \ldots, s_{T-1}, c_{T-1}, s_T) \,|\, c_0, \theta_1, \ldots, \theta_{T-1}\right]$$

$$\text{s.t.} \quad c_t = c(t, s_t, \theta_t), \qquad t = 1, \ldots, T-1,$$

$$s_{t+1} = \psi_{t+1}(s_t, c_t, z_{t+1}), \qquad 0 \le t \le T-1, \qquad s_0 \text{ is given.}$$

- This formulation is handled by the second MMCC algorithm, designed for the more general non-time-separable case.
- The objective can depend on the entire trajectory, not only on period-by-period utilities.

The MMCC Algorithm

MMCC in the time-separable case

We consider a finite-horizon stochastic control problem:

$$s_{t+1} = \psi_{t+1}(s_t, c_t, z_{t+1}), \qquad t = 0, \dots, T-1,$$

with controls parameterized by neural networks:

$$c_t = c(t, s_t, \theta_t).$$

In the time-separable case, the objective is: $\mathbb{E}\left[\sum_{t=0}^{T-1} u_{t+1}(s_{t+1}, s_t, c_t)\right]$.

Principle

At iteration k, MMCC updates the parameter vector

$$x^k = (c_0^k, \theta_1^k, \dots, \theta_{T-1}^k)$$

by alternating:

1. forward Monte Carlo simulation,
2. backward optimization of the policies,
3. update of the initial control.

The MMCC Algorithm

MMCC in the time-separable case

Starting from

$$x^{k-1} = (c_0^{k-1}, \theta_1^{k-1}, \dots, \theta_{T-1}^{k-1}),$$

the algorithm proceeds as follows.

Step A: Forward simulation

Simulate N trajectories under the current policy:

$$c_{t,l}^k = c(t, s_{t,l}^k, \theta_t^{k-1}), \qquad s_{t+1,l}^k = \psi_{t+1}(s_{t,l}^k, c_{t,l}^k, z_{t+1,l}).$$

The N paths are split into minibatches:

$$N = mb.$$

Step B: Backward policy update

For $t = T-1, \dots, 1$, update θ_t by maximizing the continuation objective:

$$\max_{\theta_t} \mathbb{E}_0 \left[\sum_{j=t}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \,\middle|\, c_0^{k-1}, \theta_1^{k-1}, \dots, \theta_{t-1}^{k-1}, \theta_t, \theta_{t+1}^k, \dots, \theta_{T-1}^k \right].$$

In practice, this expectation is approximated on each minibatch and optimized with **Adam**.

Step C: Initial control update

Once $\theta_1^k, \ldots, \theta_{T-1}^k$ have been updated, optimize

$$\max_{c_0} \mathbb{E}_0 \left[\sum_{j=0}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \,\middle|\, c_0, \theta_1^k, \ldots, \theta_{T-1}^k \right].$$

This yields the new parameter vector

$$x^k = (c_0^k, \theta_1^k, \ldots, \theta_{T-1}^k).$$

Takeaway

MMCC combines:

- **forward path simulation**,
- **backward date-by-date policy improvement**,
- **gradient optimization** with Adam.

The computational cost of one iteration is:

$$\mathcal{O}(mT^2).$$

The MMCC Algorithm

Application to Asian option hedging

We apply MMCC to the dynamic hedging of an **Asian call option** with:

- **stochastic volatility** (Heston model),
- **proportional transaction costs**,
- a **path-dependent payoff**.

Under Heston, the stock and variance evolve as

$$dS_t = rS_t\,dt + \sqrt{V_t}S_t\,dW_t,$$

$$dV_t = \kappa(\theta - V_t)\,dt + \xi\sqrt{V_t}\left(\rho\,dW_t + \sqrt{1-\rho^2}\,d\widetilde{W}_t\right)$$

The payoff is

$$g(\bar{S}_T) = \left(\bar{S}_T - K\right)^+, \qquad \bar{S}_T = \frac{1}{T}\int_0^T S_t\,dt.$$

Main challenges

- The payoff depends on the whole trajectory of the underlying, so the problem is naturally path-dependent.
- Because the option expires at a finite maturity date, the hedging problem is naturally formulated over a finite time horizon.
- Even in our setting, the augmented state $(S_t, V_t, A_t, Q_{t-1}, B_t)$ is already too rich for tractable Bellman/Grid approach, since discretizing jointly these variables leads to a rapide curse of dimensionality.

Specification of the Application

State variables and hedging objective

To embed path dependence into the state, we introduce

$$A_0 = 0, \qquad dA_t = S_t\,dt, \qquad \text{so that} \qquad \bar{S}_T = \frac{A_T}{T}.$$

The portfolio is described by:

- Q_t: stock position,
- B_t: cash account,
- Y_t: portfolio value.

With proportional transaction cost λ,

$$\Lambda_t = \lambda S_t |dQ_t|, \qquad dB_t = rB_t\,dt - S_t\,dQ_t - \Lambda_t.$$

The hedging problem is:

$$\min_{y,Q(\cdot)} \mathbb{E}\Big[\big(Y_T - g(\bar{S}_T)\big)^2\Big].$$

We seek the initial premium and the dynamic strategy that minimize the terminal quadratic hedging error.

Specification of the Application

Discrete-time MMCC formulation

We discretize the horizon:

$$0 = t_0 < t_1 < \cdots < t_N = T.$$

The control problem becomes

$$\min_{(y, q_0, \theta_{t_1}, \ldots, \theta_{t_{N-1}})} \mathbb{E}\Big[\big(Y_T - g(\bar{S}_T)\big)^2\Big].$$

At each date t_n, the rebalancing rule is approximated by a neural policy:

$$Q_{t_n} = c(t_n, s_n, \theta_{t_n}),$$

with augmented state

$$s_n = (S_{t_n}, V_{t_n}, A_{t_n}, Q_{t_{n-1}}, B_{t_n}).$$

The dynamics can then be written recursively as

$$s_{n+1} = \psi_{n+1}(s_n, Q_{t_n}, z_{n+1}).$$

Key point: Thanks to the augmented state, the path-dependent hedging problem fits the sequential MMCC framework.

Specification of the Application

Neural-network architecture and implementation

Following the MMCC framework, we use **one feed-forward neural network per trading date**:

$$Q_{t_n} = c(t_n, s_n, \theta_n), \qquad n = 1, \ldots, N-1,$$

where the input state is

$$\bar{s}_n = (\bar{S}_n, \bar{V}_n, \bar{A}_n, \bar{q}_{n-1}, \bar{B}_n) \in \mathbb{R}^5, \quad \text{where} \quad \bar{X} \text{ is the normalized value of } X$$

and the output is the scalar stock position $Q_{t_n} \in \mathbb{R}$.

- Input layer: 5 neurons
- Two hidden layers: 70 neurons each
- Output layer: 1 neuron
- Activation function: ReLU

Specification of the Application

Numerical Setup

- Initial spot price: $S_0 = 100$
- Initial variance: $V_0 = 0.0225$
- Risk-free rate: $r = 5\%$
- Maturity: $T = 1$
- Number of time steps: $N = 12$
- Strike: $K = 100$
- Transaction cost coefficient: $\lambda = 0.001$

Specification of the Application

Training Procedure

MMCC method:

- 10 outer iterations
- 30,000 Monte Carlo paths per iteration
- Mini-batch size: 4096
- 300 Adam iterations per policy
- 300 Adam iterations for the initial control (y, Q_0)
- Learning rate: 7×10^{-3}

Baseline method:

- 150 training iterations

Evaluation Criterion: The performance is evaluated on 100,000 independent Monte Carlo paths.

The criterion is the mean squared hedging error:

$$\mathbb{E}\left[\left(Y_T - g(\bar{S}_T)\right)^2\right].$$

Results

Out-of-Sample MMCC Performance

Method	Mean	Std dev	MSE	Q5%	Q50%	Q95%
MMCC	-0.0010	1.6603	2.7567	-2.9212	0.1544	2.3743

- The mean error is close to zero.
- The hedge is almost unbiased.
- The MSE remains relatively low despite transaction costs.

Results

Convergence Interpretation

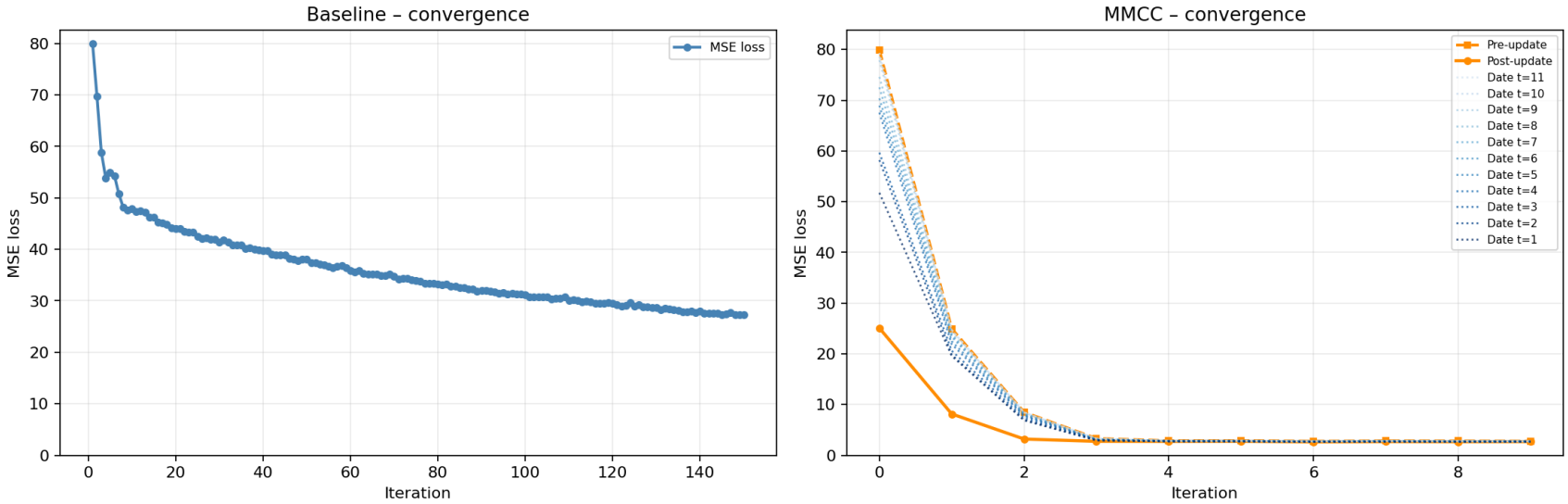

Figure: Convergence of the baseline and MMCC algorithms.

- The baseline method converges slowly.
- It does not reach a satisfactory MSE.
- The MMCC algorithm rapidly decreases the error.
- MMCC reaches an MSE below 3 after only 3 iterations.

Results

Initial Price and Initial Hedge

The MMCC algorithm gives:

$$y_0 = 5.6799, \qquad Q_0 = 0.5512.$$

- y_0 corresponds to the initial option price.
- Q_0 corresponds to the initial hedging position.

Results

Evolution of the Hedging Strategy

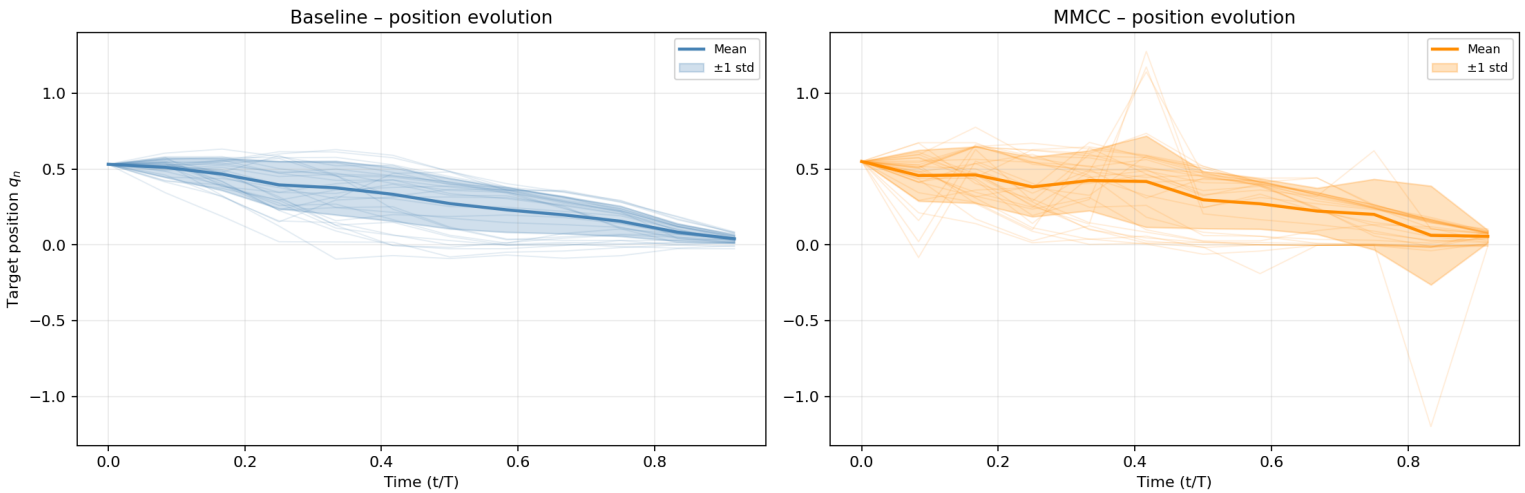

Figure: Evolution of the policy decisions Q_{t_n}.

- The baseline method produces smooth and homogeneous trajectories.
- These trajectories do not adapt sufficiently to market changes.
- MMCC produces more variable and responsive trajectories.
- This suggests a more realistic dynamic hedging strategy.

Results

Convergence Interpretation (T = 1 month)

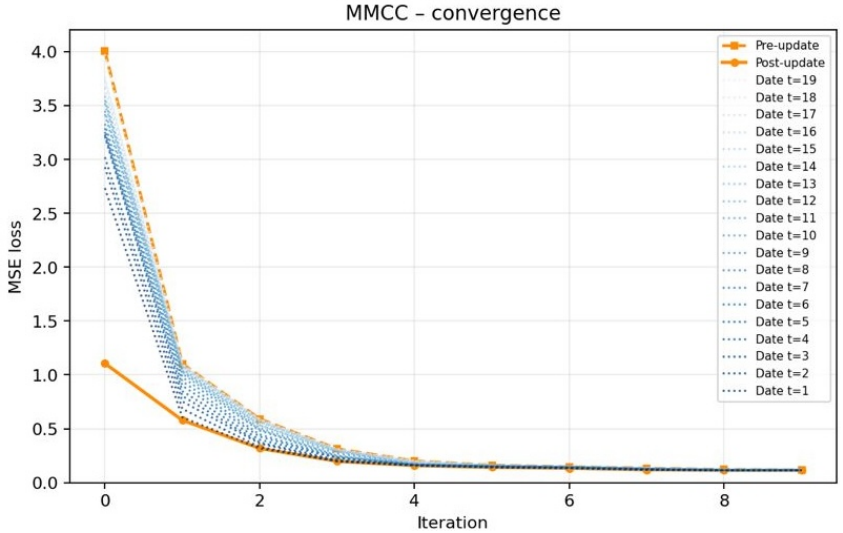

Figure: Convergence of the MMCC algorithm (T = 1 month).

- We consider a more realistic numerical configuration.
- The overall MSE is significantly smaller than in the previous experiments.
- The MMCC algorithm reaches an MSE below 0.2 after only 3 iterations, demonstrating a fast convergence.

Results

Convergence Interpretation (T = 1 month)

Method	Mean	Std dev	MSE	Q5%	Q50%	Q95%
MMCC	0.0020	0.3368	0.1135	−0.5587	0.0102	0.5417

- The mean error is close to zero.
- The hedge is almost unbiased.
- The MSE remains relatively low despite transaction costs and the maturity $T = 1$ month.

Conclusion

- MMCC provides an efficient approach for high-dimensional finite-horizon control problems.
- It performs well on the Asian option hedging problem.
- The hedging error is almost unbiased.
- Transaction costs prevent perfect hedging, making the market incomplete.